Lawful Composition Through Monadic Structures

Series: Mathematical Foundations for Universal Systems

Carolina Johnson (CJ)

Date: July 2026

Abstract

This paper examines four compositional structures within the Harmonic Framework and verifies that each admits representation as an established monadic structure from category theory. The constructions correspond respectively to the Reader, Writer, State, and Graded monads, each with its own type signature, composition rule, and operational interpretation. Explicit definitions of return and bind are provided, followed by direct proofs of the three monad laws: left identity, right identity, and associativity.

The structures analyzed originate from four distinct papers: the Doppler-Smith Correspondence, the Taylor-Doppler Identity, the Law of Admissibility, and the Blossom Harmonic Algorithm. Their original purpose was not the construction of monads, but the analysis of field sampling, phase accumulation, recursive evolution, and tiered shell composition within their respective domains. This paper demonstrates that these constructions admit established categorical formulations and satisfy the corresponding laws of composition under explicit definitions.

The contribution is therefore not the introduction of new categorical structures, but the formal verification that distinct mathematical constructions examined in this work admit representation as established compositional structures. This provides an external, independently checkable criterion for compositional consistency across the framework's domains, including acoustics, RF engineering, phase dynamics, recursive systems, and quantum computation.

1 Introduction

1.1 Motivation

Category theory provides mathematical structures for reasoning about composition. Among its most widely used abstractions are monads, which describe how computations compose while preserving well-defined algebraic laws. Their applications span functional programming, formal verification, compiler design, distributed systems, and categorical semantics.

Several mathematical constructions developed within the Harmonic Framework exhibit recurring patterns of composition. These constructions were originally developed to solve domain-specific problems rather than to satisfy categorical definitions.

The purpose of the present work is to determine whether these distinct mathematical constructions admit representation as established monadic structures. This determination is made by explicit construction rather than by analogy.

1.2 What a Monad Is

A monad consists of an endofunctor (type constructor) together with two operations, conventionally named return and bind, satisfying three laws: left identity, right identity, and associativity [1].

A structure is classified as a monad by satisfying these laws under its own type signature. Different monads need not represent the same kind of computation, nor share the same operational interpretation. What they share is lawful composition.

This paper relies exclusively on these established definitions.

1.3 Scope

Four compositional structures are examined.

Structure	Source	Computational Role	Domain
Reader	Doppler-Smith Correspondence [2]	Field sampling	Acoustics, RF Engineering
Writer	Taylor-Doppler Identity [3]	Phase accumulation	Phase dynamics
State	Law of Admissibility [4]	Recursive state evolution	Recursive systems
Graded	Blossom Harmonic Algorithm [5]	Tiered shell composition	Quantum computation

For each construction this paper defines the corresponding type constructor, defines return, defines bind, proves the three monad laws by direct substitution, and interprets the resulting composition within its original domain.

1.4 Contribution

This paper does not introduce new monadic structures, nor does it propose new category-theoretic definitions. Likewise, it does not claim that the cited mathematical constructions were originally developed as monads.

Instead, it proves that distinct mathematical constructions examined in this work admit established categorical formulations under explicit definitions of return and bind, and satisfy the corresponding monad laws.

The result is a formally verifiable criterion for compositional consistency, independently checkable using established mathematical standards.

1.5 What This Paper Does Not Claim

This paper does not claim that Reader, Writer, State, and Graded monads are equivalent structures. Each has its own type signature and computational interpretation.

It does not claim that every construction within the Harmonic Framework is a monad.

It does not claim the discovery of monads or new category-theoretic principles. Reader, Writer, State, and Graded monads are standard structures in category theory and functional programming.

It does not claim that the physical constructions themselves are inherently monadic. It claims that, under the explicit definitions given in each section, they admit a representation that satisfies the monad laws.

Finally, it does not replace the mathematical developments presented in the cited papers. Those works establish the underlying constructions. The present paper formalizes their compositional behavior within the framework of established category theory.

2 The Reader Monad: Field Sampling

2.1 Setup

From the Doppler-Smith Correspondence [2], the phase manifold M is parameterized as:

$$M = \{(x, y, z, t) \in \mathbb{R}^3 \times \mathbb{R}^+\} \tag{1}$$

with scalar phase field $\Phi : M \to \mathbb{R}$. An observer's trajectory is $r(t)$, and the observed phase is:

$$\Phi_{\text{obs}}(t) = \Phi(r(t), t) \tag{2}$$

2.2 Formal Definition

The field sampling operation is represented as a Reader computation, where the manifold coordinates provide the environment. Let the environment be $E = M$. Define the type constructor:

$$T_{\text{Reader}}(A) = E \to A \tag{3}$$

A value of type $T_{\text{Reader}}(A)$ is a function that, given an environment, produces a value of type A. Under this representation, the field $\Phi : M \to \mathbb{R}$ is an element of $T_{\text{Reader}}(\mathbb{R})$.

Return: $\text{return}(x) = \lambda E.\, x$

Bind: $m \gg\!= f = \lambda E.\, (f(m(E)))(E)$

2.3 Proof of the Laws

Left identity. $\text{return}(x) \gg\!= f = \lambda E.\, (f(\text{return}(x)(E)))(E) = \lambda E.\, (f(x))(E) = f(x)$, since $\text{return}(x)(E) = x$ for all E. ■

Right identity. $m \gg\!= \text{return} = \lambda E.\, (\text{return}(m(E)))(E) = \lambda E.\, m(E) = m$, since $\text{return}(y)(E) = y$ for all E. ■

Associativity. For any environment E: $((m \gg\!= f) \gg\!= g)(E) = g((f(m(E)))(E))(E)$. $(m \gg\!= (\lambda x.\, f(x) \gg\!= g))(E) = (f(m(E)) \gg\!= g)(E) = g((f(m(E)))(E))(E)$. Both sides are equal for all E. ■

2.4 Interpretation and Domain Implication

$\Phi_{\text{obs}}(t) = \Phi(r(t), t)$ is the application of the Reader value Φ to the environment $(r(t), t)$ under this representation. The field is read, not modified; no state carries forward; no output accumulates.

Domain implication: This formalizes field sampling in acoustics and RF engineering. The observer trajectory samples the field lawfully, meaning compositions of sampling operations are predictable and repeatable.

3 The Writer Monad: Phase Accumulation

3.1 Setup

From the Taylor-Doppler Identity [3], the phase field's Taylor expansion accumulates contributions term by term:

$$\Delta\Phi = \sum_n \frac{d^n\Phi}{dx^n}\frac{\Delta x^n}{n!} \tag{4}$$

Each term adds to a running total. This is the shape of accumulation, not sampling. The Taylor accumulation can be represented using a Writer structure.

3.2 Formal Definition

Define the type constructor $T_{\text{Writer}}(A) = A \times W$, where W is a monoid under addition. Phase may be tracked as $W = \mathbb{R}$ (absolute accumulation) or $W = \mathbb{R}/2\pi\mathbb{Z}$ (accumulation modulo 2π); both are monoids under addition, and the proofs below hold under either choice. The identity element is 0.

Return: $\text{return}(a) = (a, 0)$

Bind: $(a, w_1) \gg\!= f = \textbf{let } (b, w_2) = f(a) \textbf{ in } (b, w_1 + w_2)$

3.3 Proof of the Laws

Left identity. $\text{return}(a) \gg\!= f = \textbf{let } (b, w_2) = f(a) \textbf{ in } (b, 0 + w_2) = (b, w_2) = f(a)$. ■

Right identity. $(a, w) \gg\!= \text{return} = \textbf{let } (b, w_2) = \text{return}(a) = (a, 0) \textbf{ in } (a, w + 0) = (a, w)$. ■

Associativity. Both $((a, w_1) \gg\!= f) \gg\!= g$ and $(a, w_1) \gg\!= (\lambda x.\, f(x) \gg\!= g)$ reduce, by associativity of addition on W, to the same triple $(c, w_1 + w_2 + w_3)$, where w_2 and w_3 are the phase contributions from f and g respectively. ■

3.4 Interpretation and Domain Implication

Under this representation, each term of the Taylor expansion is a Writer computation contributing a value and a phase increment. Binding successive terms sums the increments while carrying the running value forward.

Domain implication: This formalizes phase accumulation in phase dynamics. Associativity guarantees that the total accumulated phase does not depend on how successive terms are grouped during composition.

4 The State Monad: Recursive Evolution

4.1 Setup

From the Law of Admissibility [4], recursive systems evolve by $s_{n+1} = f(s_n)$, with harmonic friction $F = |n/4k - 1| \times 100\%$ measuring deviation from the admissibility lock at $n = 4k$.

4.2 Formal Definition

Define the type constructor $T_{\text{State}}(A) = S \to (A \times S)$.

Return: $\text{return}(a) = \lambda s.\,(a, s)$

Bind: $m \gg\!= f = \lambda s.\,\textbf{let}\ (a, s') = m(s)\ \textbf{in}\ f(a)(s')$

4.3 Proof of the Laws

Left identity. $\text{return}(a) \gg\!= f = \lambda s.\,\textbf{let}\ (a, s') = (a, s)\ \textbf{in}\ f(a)(s') = \lambda s.\,f(a)(s) = f(a)$. ■

Right identity. $m \gg\!= \text{return} = \lambda s.\,\textbf{let}\ (a, s') = m(s)\ \textbf{in}\ \text{return}(a)(s') = \lambda s.\,\textbf{let}\ (a, s') = m(s)\ \textbf{in}\ (a, s') = \lambda s.\,m(s) = m$. ■

Associativity. Both sides reduce, by threading the state through identically, to $\lambda s.\,\textbf{let}\ (a, s_1) = m(s),\ (b, s_2) = f(a)(s_1),\ (c, s_3) = g(b)(s_2)\ \textbf{in}\ (c, s_3)$. ■

4.4 Interpretation and Domain Implication

Under this representation, each recursive step reads the current state and produces both a value and an updated state, matching the State monad's shape. F itself is a scalar computed from the state at a given step; it is not part of the monadic structure, it is a diagnostic read from within it.

Domain implication: This formalizes recursive evolution in physics. Associativity guarantees that a sequence of state transitions produces the same final state regardless of how the sequence is grouped when composed. It does not imply the transitions can be reordered; State's bind is sequential by construction, and later steps depend on the state produced by earlier ones.

5 The Graded Monad: Tiered Shell Composition

5.1 Setup

The Blossom Harmonic Algorithm [5] composes computations across harmonic shells indexed by tier K, with return embedding a value at the base shell and bind advancing composition to the next tier while accumulating geometric phase:

$$\gamma_{K+1} = \gamma_K + \gamma_K(C) \tag{5}$$

The categorical operations defined below formalize the existing tier advancement and phase accumulation rules used by [5]; they do not introduce a new computational mechanism. Return

corresponds to embedding a value at the base tier with zero accumulated phase, and bind corresponds to a tier transition that validates and advances the shell index while accumulating γ. BHA admits a graded monadic formulation whose effect combines tier advancement with additive phase accumulation [6, 7]: a family of type constructors T_g indexed by a monoid, here $(\mathbb{N}, +, 0)$, tier number under addition. Return targets the identity grade:

Return: $\text{return} : A \to T_0(A)$

Bind, general form: $\text{bind} : T_m(A) \to (A \to T_n(B)) \to T_{m+n}(B)$

Composition may advance by any grade n; a single-tier step is the case $n = 1$.

5.2 Formal Definition

Let a graded value be represented as a triple (a, K, γ), with tier $K \in \mathbb{N}$ and phase $\gamma \in \mathbb{R}$, accumulating additively.

Return: $\text{greturn}(a) = (a, 0, 0)$

Bind: $\text{gbind}((a, K, \gamma), f) = \textbf{let } (b, n, \gamma_{\text{step}}) = f(a) \textbf{ in } (b, K + n, \gamma + \gamma_{\text{step}})$

5.3 Proof of the Laws

Left identity. $\text{gbind}(\text{greturn}(a), f) = \textbf{let } (b, n, \gamma_{\text{step}}) = f(a) \textbf{ in } (b, 0+n, 0+\gamma_{\text{step}}) = (b, n, \gamma_{\text{step}}) = f(a)$. ■

Right identity. $\text{gbind}((a, K, \gamma), \text{greturn}) = \textbf{let } (b, n, \gamma_{\text{step}}) = \text{greturn}(a) = (a, 0, 0) \textbf{ in } (a, K + 0, \gamma + 0) = (a, K, \gamma)$. ■

Associativity. Let $f(a) = (b, n, \gamma_f)$ and $g(b) = (c, p, \gamma_g)$.

$\text{gbind}(\text{gbind}((a, K, \gamma), f), g) = \text{gbind}((b, K + n, \gamma + \gamma_f), g) = (c,\ K + n + p,\ \gamma + \gamma_f + \gamma_g)$

$\text{gbind}((a, K, \gamma), \lambda x.\, \text{gbind}(f(x), g)) = \text{gbind}((a, K, \gamma), h)$ where $h(a) = \text{gbind}((b, n, \gamma_f), g) = (c,\ n + p,\ \gamma_f + \gamma_g) = (c,\ K + (n + p),\ \gamma + (\gamma_f + \gamma_g)) = (c,\ K + n + p,\ \gamma + \gamma_f + \gamma_g)$

Both reduce to the same triple. ■

5.4 Interpretation and Domain Implication

BHA's original formulation composes tier, validation, advancement, and phase accumulation; the graded monad defined above composes return, bind, grade, and phase accumulation. These represent the same underlying computation: return corresponds to tier initialization, bind's grade-advancing step corresponds to validated tier advancement, and γ accumulates identically in both formulations. This representation is verified above by direct substitution. BHA's physical zero-drift gate and the correspondence to Berry phase are separate structural questions addressed in [5].

Domain implication: This formalizes tiered shell composition in quantum computation. Associativity guarantees that a sequence of tier transitions produces the same final tier and accumulated phase regardless of how the sequence is grouped when composed. It does not imply the transitions commute; reordering which transition is applied first is physically distinct, consistent with the non-abelian character of holonomic phase under general paths.

6 Discussion

6.1 What These Four Results Show

Four structures within the Harmonic Framework, field sampling, phase accumulation, recursive state evolution, and tiered shell composition, each admit representation satisfying the monad laws under explicit, verified definitions of return and bind. The fourth is formalized using a graded index, following the established graded monad framework [6, 7], with bind generalized from a fixed tier increment to an arbitrary graded composition.

These patterns are not unique to this framework. Reader, Writer, State, and graded structures are standard in category theory and functional programming. They appear in any system that exhibits the corresponding shape of computation. The value of this paper is not the discovery of these patterns but the verification that the distinct mathematical constructions examined here, each built for a different purpose, admit representations that satisfy them exactly.

6.2 Why This Matters for the Framework's Domains

Each paper in the framework addresses a distinct physical or computational question: acoustics and RF (Doppler-Smith), phase dynamics (Taylor-Doppler), recursive physics (Law of Admissibility), and quantum computation (BHA). That each construction admits a representation satisfying the same compositional laws means the framework's internal logic is consistent across domains.

This provides a checkable standard for future work in any of these domains. A claim that a new structure in the framework composes correctly can be checked against the same three laws used here, independent of which physical domain it comes from. The framework's composition rules are not just analogies; they are formally testable.

7 Conclusion

This paper has shown that the constructions examined admit established monadic formulations satisfying the monad laws by direct proof. The result is an externally verifiable criterion for compositional consistency expressed through established category-theoretic structures. Under those definitions, the constructions presented here admit lawful composition.

Appendix A: Computational Verification

Each of the four monad law sets was additionally verified by direct numerical substitution, independent of the symbolic proofs in Sections 2 through 5, using concrete instantiated functions and values. These examples verify particular instantiations of the algebraic definitions given above and do not replace the general proofs.

A.1 Reader (Environment $E = (2.0, 4.0)$, test value $a = 5.0$)

Test functions: $\Phi(r, t) = r \cdot t$ (the Reader value under test); $f(x) = \lambda E.\, x + \Phi(E)$; $g(y) = \lambda E.\, y \times 2 + E_r$, where E_r denotes the first component of E. For right identity and associativity, $m = \Phi$.

Law	Left-hand side	Right-hand side	Equal
Left identity	13.0	13.0	✓
Right identity	8.0	8.0	✓
Associativity	34.0	34.0	✓

A.2 Writer (Test value $a = 5.0$, prior value $w = 1.5$)

Test functions: $f(x) = (x + 1,\ 2.0)$; $g(y) = (y \times 2,\ 3.0)$; $m = (5.0, 1.5)$ for right identity and associativity.

Law	Left-hand side	Right-hand side	Equal
Left identity	(6.0, 2.0)	(6.0, 2.0)	✓
Right identity	(5.0, 1.5)	(5.0, 1.5)	✓
Associativity	(12.0, 6.5)	(12.0, 6.5)	✓

A.3 State (Initial state $s_0 = 3.0$)

Test functions: $f(x) = \lambda s.\, (x + s,\ s + 1)$; $g(y) = \lambda s.\, (y \times s,\ s + 2)$; $m = \lambda s.\, (s \times 10,\ s + 1)$; test value $a = 7.0$ for left identity.

Law	Left-hand side	Right-hand side	Equal
Left identity	(10.0, 4.0)	(10.0, 4.0)	✓
Right identity	(30.0, 4.0)	(30.0, 4.0)	✓
Associativity	(170.0, 7.0)	(170.0, 7.0)	✓

A.4 Graded (Initial value $a_0 = 5.0$, tier $K = 3$, phase $\gamma = 1.2$)

Test functions: $f(x) = (x + 1,\ 1,\ 0.3)$; $g(y) = (y \times 2,\ 2,\ 0.7)$; $m = (5.0, 3, 1.2)$ for right identity and associativity.

Law	Left-hand side	Right-hand side	Equal
Left identity	(6.0, 1, 0.3)	(6.0, 1, 0.3)	✓
Right identity	(5.0, 3, 1.2)	(5.0, 3, 1.2)	✓
Associativity	(12.0, 6, 2.2)	(12.0, 6, 2.2)	✓

References

[1] Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1), 55–92.

[2] Johnson, C. (2026). The Doppler-Smith Correspondence: Shared Möbius Structure in Impedance and Frequency Ratios. SemanticShift.

[3] Johnson, C. (2026). Taylor-Doppler Identity: Frequency Transport on a Phase Manifold. SemanticShift.

[4] Johnson, C. (2025). Law of Admissibility: Structural Invariants in Recursive Dynamics (R = 4). SemanticShift.

[5] Johnson, C. (2025). Blossom Harmonic Algorithm as Holonomic Monad: A Unified Framework for Topologically Protected Quantum Computation. SemanticShift.

[6] Atkey, R. (2009). Parameterised Notions of Computation. *Journal of Functional Programming*, 19(3-4), 335–376.

[7] Katsumata, S. (2014). Parametric Effect Monads and Semantics of Effect Systems. *POPL '14*.